

Design Notes

Overall solution design

The solution is a terminal-based conversational assistant that orchestrates a luggage workflow end to end: validate booking, fetch eligible luggage options, confirm user intent, and submit luggage additions. The conversation loop is managed in *main.py*, API integration lives in *api.py*, and transient conversation state is centralized in *state.py*.

Key architectural decisions

I separated logic into modules: orchestration and tool-call handling in *main.py*, external HTTP calls in *api.py*, tool definitions and escalation triggers in *tools.py*, and utility functions in *utils.py*.

I also used **real-time dynamic tool creation** for *add_luggage* in *tools.py*, so the model can only choose valid passenger IDs and luggage IDs for the given booking reference. The tool is also created again after the user try to add a luggage to the booking to inform the agent of the updated luggage availability.

To improve response quality and message understanding, I implemented **confirmation detection using an LLM** instead of keyword matching. This makes the assistant more robust to natural language variations, where rule-based keyword matching is less robust.

To allow users to **manage luggage for multiple flights**, I changed the booking-reference handling: the global variable *current_luggage_options* is a booking-reference-based list/dictionary, and the current booking reference is saved in the global variable *current_booking_reference* and updated whenever the user changes it.

To allow users to **add multiple bags for multiple passengers in one go**, the *add_luggage* function accepts a list of luggage items and loops through them, calling the *add-luggage* API for each item.

I refactored the codebase to make it flatter and simpler, avoiding deeply nested logic, and I used clear, friendly variable and function names without abbreviations to improve readability.

I refactored the code by splitting large functions into smaller, single-purpose functions to improve readability, maintainability, and testability.

I used global variables to maintain conversation state across multiple agent iterations. This is critical to keep baggage availability up to date after the user performs actions such as adding luggage.

How the assistant uses the provided APIs

The assistant first checks service health, then calls booking lookup, then luggage options, and finally add-luggage when the user confirms. API payload mapping and response normalization are handled in *api.py*, while tool execution order and conversation-state updates are coordinated in *main.py*. Escalation requests are sent through the escalation endpoint via *tools.py* and *api.py*.

API calls are made only when required, for example escalation is triggered only once.

Error handling and escalation strategy

The project uses layered error handling: API **exceptions are caught and logged**, user-facing errors are returned as structured messages, and repeated/critical failures trigger human escalation. Examples include escalation when the API is unreachable, OpenAI call failures, **booking lookup after three repeated failures**, and baggage payload/response mismatch.

Human escalation is implemented through tools and is not triggered immediately. The agent first asks the user for confirmation, so the tool call requires a *confirmation_or_cancellation* argument before escalation can proceed. **Escalation can also be triggered programmatically**: to achieve this, a fake tool-call initialization message is injected into the messages array sent to OpenAI.

Passing only the last user message during human escalation is unreliable, because it is often just a confirmation (for example, OK). The previous message may also not contain the real issue that triggered escalation. A better approach is to send the **full conversation transcript** so human agents have complete context.

Assumptions and trade-offs

The design assumes a single interactive session with in-memory state only, which keeps implementation simple but is **not durable across restarts**. It prioritizes safe operational flow over broad scope by constraining the assistant to luggage and escalation tasks.

Another trade-off is limited resilience features (for example, **no retry/backoff** strategy on most endpoints), accepting simplicity in exchange for faster implementation and readability.

OpenAI responses are currently handled via standard REST requests; using **streaming responses** instead would reduce user-perceived waiting time and improve the chat experience.

The *luggage-options* API is called only the first time a user asks to manage baggage. After that, *current_luggage_options* data is used **locally**. I chose this because the mock APIs do not persist real booking updates, so state changes must be maintained in memory.

Trade-off: this approach can cause consistency issues in production when multiple users (for example, customers on the same booking) modify baggage at the same time from different devices.

Luggage labels include direction only when multiple directions exist, and **omit** **outbound/return** when there is only one direction for simpler responses and user input. Trade-off: better usability, but less explicit detail in edge cases where full direction context may be useful.